

**ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА**

**ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №3
«Ітератори, генератори та потокова обробка даних»
з курсу «Професійний Python»
Варіант №1 — «Потокова обробка результатів студентів»**

Виконав: Лис Артур

Група: ФЕП-31

Перевірів(ла): _____

Дата: 08.09.2026

Львів — 2026

1. Мета роботи

Навчитися працювати з ітераційною моделлю Python не як з магією циклу `for`, а як з протоколом, який можна реалізувати руками: `iterable` та `iterator`, `__iter__()` / `__next__()`, `StopIteration`. На цій основі побудувати `generator functions` з `yield` та `yield from`, `generator expressions`, застосувати модуль `itertools` і зібрати з окремих лінивих стадій потоковий конвеєр, який обробляє сотні тисяч записів CSV, не завантажуючи файл у пам'ять. Експериментально порівняти `eager` та `lazy` реалізації за часом, піковою пам'яттю та часом до першого результату.

2. Індивідуальний варіант і постановка задачі

Варіант №1 — «Потокова обробка результатів студентів». Вхід: CSV-файл із колонками `student_id, name, group, grade`. Потрібно реалізувати потокове читання CSV, валідацію та відкидання некоректних оцінок, відбір студентів із балом не нижче порога, обчислення середнього, мінімуму й максимуму, перші `N` найкращих валідних записів, `batching` і підрахунок записів за групами.

Загальні вимоги роботи: джерело даних; окремі генератори для читання, очищення, парсингу, фільтрації та трансформації; стадія агрегації; `generator expression`; щонайменше три функції з `yield`, одна конструкція `yield from`, три засоби `itertools`, `islice`; `batching`; не менше 100 000 записів; порівняння `eager` і `lazy` варіантів з оцінкою пам'яті. Із завдань підвищеної складності реалізовано власний `iterator class`, рекурсивний генератор із `yield from`, нескінченний генератор + `islice`, `chain` для кількох файлів, `accumulate`, `pairwise`, `groupby`, динамічний `batching`, потоковий експорт у новий файл, підрахунок `invalid records` за причинами, `logging` стадій, вимірювання `time-to-first-result`, `lazy search` із раннім завершенням і порівняння пам'яті `list` проти `generator`.

3. Вхідний набір даних

Реальних 100 000 результатів у мене немає, тому дані генеруються. Модуль `generate.py` — теж генератор: `student_ids()` нескінченно рахує ідентифікатори поверх `itertools.count()`, `generate_rows()` обрізає його через `islice` і видає рядки по одному, `write_dataset()` пише їх у CSV, тримаючи в пам'яті один рядок. Кожен 97-й рядок навмисно зіпсовано (чотири види дефектів по черзі: текст замість оцінки, оцінка 102.5, порожнє ім'я, пропущена колонка), кожен 13-й — валідний, але «брудний»: зайві пропуски й малі літери. Прізвища та імена — з набору даних ЛР2, групи — ФЕП. Той самий `seed` завжди дає той самий файл, тож результати відтворювані.

```
def student_ids(start: int = 1) -> Iterator[int]:
    """Infinite stream of identifiers on top of itertools.count(); always cut it with islice."""
    yield from count(start)

def generate_rows(rows: int, seed: int = 3) -> Iterator[list[str]]:
    """Lazily yield `rows` CSV rows; the same seed always gives the same file."""
    rng = random.Random(seed)
    for student_id in islice(student_ids(), rows):
        name = f"{rng.choice(LAST_NAMES)} {rng.choice(FIRST_NAMES)}"
```

```

group = rng.choice(GROUPS)
grade = f"{rng.triangular(40, 100, 82):.1f}"
row = [str(student_id), name, group, grade]
if student_id % DEFECT_EVERY == 0:
    yield _break(row, student_id // DEFECT_EVERY)
elif student_id % DIRTY_EVERY == 0:
    yield [row[0], " " + name.lower().replace(" ", " "), group.lower(), grade]
else:
    yield row

```

Для демонстрації генерується 200 000 рядків (9.4 MB), для експерименту — 10 000, 100 000, 500 000 і 1 000 000. Каталог data/ не комітиться: у Git іде генератор, а не 90 MB даних.

НАБІР ДАНИХ: data\students.csv – знайдено, 200 000 рядків, 9.4 MB

```

-----
generate_rows(3) – generator, list() дає: [['1', 'Кравець Максим', 'ФЕП-33', '90.5'], ['2', 'Шевчук
Богдан', 'ФЕП-41', '45.8'], ['3', 'Шевчук Олег', 'ФЕП-41', '64.3']]
перші 5 рядків файлу (islice над read_lines):
student_id,name,group,grade
1,Кравець Максим,ФЕП-33,90.5
2,Шевчук Богдан,ФЕП-41,45.8
3,Шевчук Олег,ФЕП-41,64.3
4,Шевчук Ярина,ФЕП-34,80.1
«брудний» рядок 13 та зіпсовані рядки 97, 194, 291, 388:
13: '13, савчук соломія,феп-41,78.4'
97: '97,Гнатишин Роман,ФЕП-32,102.5'
194: '194,,ФЕП-41,76.0'
291: '291,Петришин Тарас,ФЕП-34'
388: '388,Мельник Остап,ФЕП-34,error'

```

4. Структура проєкту

Робота продовжує проєкт student-manager (ЛР1–ЛР2) новим підпакетом streaming; версія 0.3.0. Кожна стадія конвеєра живе в окремому модулі, щоб її можна було тестувати і замінювати незалежно.

```

src/
  student_manager/
    __init__.py
    cli.py
    config.py
    exceptions.py
    main.py
    models.py
    registry.py
    analytics/
      __init__.py
      __main__.py
      analytics.py
      benchmark.py
      data.py
      decorators.py
      processors.py
  streaming/
    __init__.py
    __main__.py
    aggregates.py

```

```

batching.py
experiment.py
filters.py
generate.py
models.py
parsers.py
pipeline.py
readers.py
transforms.py

tests/
  test_analytics.py
  test_registry.py
  test_streaming.py

```

Модуль	Призначення
models.py	StudentRecord — NamedTuple одного валідного рядка; малий, незмінний, дешевий
generate.py	джерело: нескінченний count + islice, потоковий запис CSV
readers.py	read_lines (generator), LineReader (iterator), CsvDataset (iterable-контейнер)
parsers.py	clean_lines, parse_rows (yield from DictReader), validate + ValidationReport
filters.py	with_min_grade, in_groups, where, head_by_id (takewhile)
transforms.py	tidy / normalize (generator expression), міст to_grade_records до ЛІР2
batching.py	batched, adaptive_batches (islice), рекурсивний flatten (yield from)
aggregates.py	streaming_stats, top_n (heapq), running_average (accumulate), grade_changes (pairwise), by_group (groupby), find_first
pipeline.py	traced (logging стадій), assemble / build_pipeline, build_multi_pipeline (chain), export_csv
experiment.py	eager проти lazy: час, peak memory (tracemalloc), time to first result, раннє завершення
__main__.py	демонстрація всього переліченого: python -m student_manager.streaming

5. Iterable та iterator: протокол без магії

Iterable — об'єкт, у якого є `__iter__()`, що повертає iterator. Iterator — об'єкт із `__next__()`, який видає елементи по одному й кидає `StopIteration`, коли їх не лишилося; його `__iter__()`

повертає self, тому iterator сам є iterable. Цикл for робить рівно це: iter(), потім next() до StopIteration. Різниця важлива на практиці: контейнер можна обійти багато разів, iterator — один раз.

LineReader реалізує протокол над файлом руками: файл відкривається при першому next(), lines_read рахує прочитані рядки, порожній рядок від readline() означає кінець — файл закривається і летить StopIteration. CsvDataset — контейнер: кожен iter() видає новий LineReader.

```
class LineReader:
    """Iterator over the lines of a file: __iter__ returns self, __next__ pulls one line, StopIteration
    closes it.

    read_lines() does the same in three lines. The class exists to show the protocol without magic and
    to
    expose `lines_read`, which lets the caller see how far an early-terminated pipeline actually got.
    """

    def __init__(self, path: Path) -> None:
        self.path = path
        self.lines_read = 0
        self._file: IO[str] | None = None
        self._done = False

    def __iter__(self) -> LineReader:
        return self

    def __next__(self) -> str:
        if self._done:
            raise StopIteration
        if self._file is None:
            self._file = self.path.open("r", encoding="utf-8", newline="")
        line = self._file.readline()
        if not line:
            self.close()
            raise StopIteration
        self.lines_read += 1
        return line

    def close(self) -> None:
        """Release the file; after this the iterator is exhausted for good (one-shot)."""
        self._done = True
        if self._file is not None:
            self._file.close()
            self._file = None

    @property
    def closed(self) -> bool:
        return self._file is None


class CsvDataset:
    """Iterable container: every iter() hands out a fresh LineReader, so the file can be walked again
    and again."""
```

```

def __init__(self, path: Path) -> None:
    self.path = path

def __iter__(self) -> LineReader:
    return LineReader(self.path)

def __repr__(self) -> str:
    return f"CsvDataset({self.path.name!r})"

ITERABLE та ITERATOR: CsvDataset (контейнер) і LineReader (iterator)
-----
iter(dataset) is dataset -> False (контейнер щоразу видає новий LineReader)
iter(reader) is reader -> True (iterator повертає себе)
next(reader) x3 -> ['student_id,name,group,grade', '1,Кравець Максим,ФЕП-33,90.5', '2,Шевчук
Богдан,ФЕП-41,45.8']
reader.lines_read = 3, файл відкрито: True
цикл for дочитав решту: 199 998 рядків; після StopIteration файл закрито: True
next(reader) ще раз -> StopIteration; list(reader) -> [] (one-shot)
next(iter(dataset)) -> 'student_id,name,group,grade' (контейнер можна обійти знову)

```

Лічильник `lines_read` не декоративний: у розділі 9 саме він показує, скільки рядків файлу реально прочитав лінійний пошук, який зупинився на першому збігу.

6. Генератори: `yield`, `yield from`, `generator expression`

6.1. Generator function і `yield`

Ті самі три рядки, що й `LineReader`, але через `yield`: функція `read_lines()` при виклику нічого не робить і повертає генератор-об'єкт; тіло виконується з першим `next()`, зупиняється на кожному `yield` зі збереженням локальних змінних і відкритого блоку `with`. Анотація `Generator` замість `Iterator` дозволяє споживачу викликати `close()` і звільнити файл раніше, ніж потік вичерпано.

```

def read_lines(path: Path) -> Generator[str, None, None]:
    """Generator: the file is opened on the first next() and stays open only while lines are being
    pulled.

    Annotated as Generator rather than Iterator so that callers may close() it and release the file
    early.
    """
    with path.open("r", encoding="utf-8", newline="") as file:
        yield from file

GENERATOR FUNCTION і yield: read_lines()
-----
read_lines(path) -> generator, стан GEN_CREATED: файл ще не відкрито
next(lines) -> 'student_id,name,group,grade', стан GEN_SUSPENDED: функція «завмерла» на yield
lines.close() -> стан GEN_CLOSED: блок with усередині генератора закрив файл

```

6.2. Нескінченний генератор та `islice`

`student_ids()` ніколи не закінчується — це `yield from count(start)`. Його можна споживати лише через `islice` або `next()`; `list()` чекав би `StopIteration` вічно.

НЕСКІНЧЕННИЙ ГЕНЕРАТОР + islice

```
list(islice(student_ids(200_001), 5)) -> [200001, 200002, 200003, 200004, 200005]
list(student_ids()) не завершився б ніколи: count() не має кінця, а list() чекає на StopIteration
```

6.3. yield from

Дві полі yield from: делегування іншому iterator (parse_rows() віддає рядки csv.DictReader, який сам лінійний і бере заголовок при першому next()) та рекурсія (flatten() розгортає вкладені списки будь-якої глибини — батчі батчів — в один плоский потік).

```
def parse_rows(lines: Iterable[str]) -> Generator[RawRow, None, None]:
    """csv.DictReader is lazy itself: it takes the header on the first next(), then one row per
    next()."""
    yield from csv.DictReader(lines)

def flatten(values: Iterable[Nested[T]]) -> Iterator[T]:
    """Recursive generator: nested lists of any depth (batches of batches) back into one flat
    stream."""
    for value in values:
        if isinstance(value, list):
            yield from flatten(value)
        else:
            yield value

yield from: parse_rows() делегує csv.DictReader, flatten() — сам собі (рекурсія)
-----
next(parse_rows(...)) -> {'student_id': '1', 'name': 'Кравець Максим', 'group': 'ФЕП-33', 'grade':
'90.5'}
flatten([1, [2, 3], [4, [5, 6]]]) -> [1, 2, 3, 4, 5, 6]
```

6.4. Generator expression

Вираз у круглих дужках — лінійний двійник list comprehension. Стадія normalize() — саме він; сума всіх валідних балів у демонстрації теж рахується без проміжного списку.

```
def normalize(records: Iterable[StudentRecord]) -> Iterator[StudentRecord]:
    """Mapping stage as a generator expression over tidy()."""
    return (tidy(record) for record in records)
```

GENERATOR EXPRESSION: сума всіх валідних балів без проміжного списку

```
grades = (record.grade for record in build_pipeline(path)) -> generator
sum(grades) = 14 647 301.3
```

7. Потоковий конвеєр

7.1. Схема

```
CSV file
├─ read_lines()      source      generator: рядок за рядком, файл відкрито лише під час обходу
├─ clean_lines()    cleaning    strip, без порожніх рядків
├─ parse_rows()     parsing     yield from csv.DictReader -> dict
├─ validate()       validation  dict -> StudentRecord, відхилені рахуються за причиною
├─ normalize()      transformation generator expression над tidy()
├─ with_min_grade() filtering   grade >= threshold
└─ sink: streaming_stats / top_n / batched / export_csv / islice
```

Жодна стадія не тримає більше одного елемента. Конвеєр збирає `assemble()`; `build_pipeline()` підставляє джерело-файл, а `build_multi_pipeline()` через `chain.from_iterable` робить один потік із кількох файлів, відкриваючи кожен лише тоді, коли до нього доходить черга. Обгортка `traced()` логує, скільки елементів пройшло стадією, — але тільки коли стадію вичерпано, бо генератор виконується рівно настільки, наскільки його тягнуть.

```
def traced(name: str, items: Iterable[T]) -> Generator[T, None, None]:
    """Logging wrapper around a stage: counts what passed and reports it once the stage is drained.

    If the consumer stops early (islice, find_first) the stage is never drained and nothing is logged:
    a generator only runs as far as it is pulled.
    """
    passed = 0
    for item in items:
        passed += 1
        yield item
    log.info("stage %-8s drained, %d items passed", name, passed)

def assemble(lines: Iterable[str], minimum: float = 0.0,
             report: ValidationReport | None = None) -> Generator[StudentRecord, None, None]:
    """Chain the stages over any source of lines; nothing runs until the consumer asks for a record."""
    rows = parse_rows(clean_lines(traced("read", lines)))
    valid = traced("validate", validate(rows, report))
    return traced("filter", with_min_grade(normalize(valid), minimum))

def build_pipeline(path: Path, minimum: float = 0.0,
                  report: ValidationReport | None = None) -> Generator[StudentRecord, None, None]:
    """The pipeline over one CSV file. The file is not even opened until the first next(); close()
    releases it."""
    return assemble(read_lines(path), minimum, report)

def build_multi_pipeline(paths: Iterable[Path], minimum: float = 0.0) -> Iterator[StudentRecord]:
    """One stream over several files: chain.from_iterable opens each file only when its turn comes."""
    return chain.from_iterable(build_pipeline(path, minimum) for path in paths)
```

7.2. Парсинг і валідація

`parse_record()` перетворює `dict` на `StudentRecord` або кидає `ValueError` із коротким кодом причини; `validate()` пропускає валідні записи далі, а невалідні рахує у `ValidationReport` за причиною й відкидає, ніколи не перериваючи потік.

```
def parse_record(row: RawRow) -> StudentRecord:
    """One raw row -> StudentRecord, or ValueError whose message is a short reason code."""
    id_text, name, group, grade_text = (_field(row, column) for column in COLUMNS)
    if not name or not group:
        raise ValueError("empty_field")
    try:
        student_id, grade = int(id_text), float(grade_text)
    except ValueError as error:
        raise ValueError("not_a_number") from error
    if not GRADE_MIN <= grade <= GRADE_MAX:
        raise ValueError("out_of_range")
    return StudentRecord(student_id, name, group, grade)
```

```
def validate(rows: Iterable[RawRow], report: ValidationReport | None = None) ->
Iterator[StudentRecord]:
    """Generator: valid rows pass through, invalid ones are counted by reason and dropped, never
    raised."""
    for row in rows:
        try:
            record = parse_record(row)
        except ValueError as error:
            if report is not None:
                report.rejected[str(error)] += 1
            continue
        if report is not None:
            report.accepted += 1
        yield record
```

7.3. Фільтрація і трансформація

```
def with_min_grade(records: Iterable[StudentRecord], minimum: float) -> Iterator[StudentRecord]:
    """Variant 1: students with grade >= threshold."""
    for record in records:
        if record.grade >= minimum:
            yield record

def head_by_id(records: Iterable[StudentRecord], last_id: int) -> Iterator[StudentRecord]:
    """Records while student_id <= last_id. Ids grow along the file, so takewhile stops reading it
    early."""
    return takewhile(lambda record: record.student_id <= last_id, records)

def tidy(record: StudentRecord) -> StudentRecord:
    """Collapse spaces and title-case the name, upper-case the group prefix («фен-31с» -> «ФЕП-
    31с»)."""
    prefix, dash, rest = record.group.partition("-")
    return record._replace(
        name=_SPACES.sub(" ", record.name.strip()).title(),
        group=f"{prefix.upper()}{dash}{rest.lower()}",
    )
```

7.4. Агрегація за один прохід

Stats.add() оновлює лічильник, суму, мінімум, максимум і Counter за групами, тому середнє, min/max і розподіл за групами (усе, що просить варіант) обчислюються одним проходом, а пікова пам'ять не залежить від кількості записів. top_n() тримає в купі лише N найкращих, find_first() зупиняє потік на першому збігу.

```
class Stats:
    """Everything variant 1 asks for, gathered in one pass without keeping a single record."""

    count: int = 0
    total: float = 0.0
    minimum: float | None = None
    maximum: float | None = None
    per_group: Counter[str] = field(default_factory=Counter)

    @property
    def average(self) -> float:
        return self.total / self.count if self.count else 0.0

    def add(self, record: StudentRecord) -> None:
```

```

        self.count += 1
        self.total += record.grade
        self.per_group[record.group] += 1
        if self.minimum is None or record.grade < self.minimum:
            self.minimum = record.grade
        if self.maximum is None or record.grade > self.maximum:
            self.maximum = record.grade

def streaming_stats(records: Iterable[StudentRecord]) -> Stats:
    """Count, average, min, max and records per group in one pass; peak memory does not depend on n."""
    stats = Stats()
    for record in records:
        stats.add(record)
    return stats

def top_n(records: Iterable[StudentRecord], n: int) -> list[StudentRecord]:
    """The n best grades of a stream: heapq keeps only n items while the whole stream flows past."""
    return heapq.nlargest(n, records, key=lambda record: (record.grade, -record.student_id))

def find_first(records: Iterable[StudentRecord], predicate: Callable[[StudentRecord], bool]) -> StudentRecord | None:
    """Lazy search: stops pulling from the stream at the first match (early termination)."""
    return next((record for record in records if predicate(record)), None)

```

LAZY PIPELINE: перші 5 записів із балом ≥ 85 (islice)

```

-----
1  Кравець Максим      ФЕП-33    90.5
5  Гнатишин Оксана    ФЕП-32    88.1
7  Бойко Остап        ФЕП-31с   86.2
14 Кравець Роман       ФЕП-31с   87.5
15 Дзюба Роман        ФЕП-41    89.9

```

жодного рядка «[log] stage ... drained»: потік зупинено після п'яти записів, файл закрито

АГРЕГАЦІЯ ЗА ОДИН ПРОХІД: streaming_stats + ValidationReport (стадії логують, коли вичерпані)

```

-----
[log] stage read      drained, 200001 items passed
[log] stage validate  drained, 197939 items passed
[log] stage filter    drained, 41535 items passed
рядків даних: 200 000; валідних: 197 939; відхилено: 2 061
empty_field          515
missing_column       515
not_a_number         515
out_of_range         516
із балом  $\geq 85$ : 41 535 записів; середній 89.96; min 85.0; max 99.9
за групами: {'ФЕП-31с': 8338, 'ФЕП-32': 8259, 'ФЕП-33': 8140, 'ФЕП-34': 8390, 'ФЕП-41': 8408}
(далі логування стадій вимкнено)

```

ТОР-5 НАЙКРАЩИХ (heapq.nlargest: у пам'яті лише п'ять записів)

```

-----
71798  Гаврилюк Тарас    ФЕП-41    99.9
197236 Скиба Дарина      ФЕП-34    99.9
4836   Шевчук Роман     ФЕП-31с   99.8
156973 Скиба Ярина       ФЕП-33    99.8
2191   Кравець Христина ФЕП-34    99.7

```

7.5. Batching

`batched()` ріже потік на порції фіксованого розміру через `islice`: у пам'яті лише поточна порція. `adaptive_batches()` подвоює розмір порції після кожного кроку до межі — як TCP slow start, коли споживача (базу, API) варто «розігріти» перед повними порціями.

```
def batched(items: Iterable[T], size: int) -> Generator[list[T], None, None]:
    """Fixed-size batches; only the current batch is materialised, the rest of the stream stays
    lazy."""
    if size < 1:
        raise ValueError("batch size must be positive")
    iterator = iter(items)
    while batch := list(islice(iterator, size)):
        yield batch
```

БАТЧІ: `batched(..., 50 000)` та `adaptive_batches(start=1 000, limit=64 000)`

batched: [50000, 50000, 50000, 47939], разом 197 939

adaptive: [1000, 2000, 4000, 8000, 16000, 32000, 64000, 6939], разом 197 939

8. Використані засоби itertools

Функція	Де застосовано	Навіщо
count	generate.student_ids	нескінченний потік ідентифікаторів
islice	generate_rows, batched, adaptive_batches, демонстрація	обрізати нескінченне, взяти перші N, вирізати порцію
chain.from_iterable	pipeline.build_multi_pipeline	один потік із кількох файлів
takewhile	filters.head_by_id	читати, поки id <= межі, і зупинити файл раніше
accumulate	aggregates.running_average	накопичувальні суми для ковзного середнього
pairwise	aggregates.grade_changes	різниця сусідніх оцінок
groupby	aggregates.by_group	групування відсортованого батчу за групою

```
def running_average(grades: Iterable[float]) -> Iterator[float]:
    """Lazy cumulative mean: accumulate() gives running sums, count() the divisors."""
    return (total / n for total, n in zip(accumulate(grades), count(1)))
```

```
def grade_changes(records: Iterable[StudentRecord]) -> Iterator[float]:
```

```

"""Difference between neighbouring grades in the stream (pairwise)."""
return (current.grade - previous.grade for previous, current in pairwise(records))

def by_group(records: Iterable[StudentRecord]) -> Iterator[tuple[str, list[StudentRecord]]]:
    """Group a batch with groupby(): it only joins neighbours, so the batch is sorted by the same key
    first."""
    ordered = sorted(records, key=lambda record: record.group)
    for group, members in groupby(ordered, key=lambda record: record.group):
        yield group, list(members)

groupby НА ОДНОМУ БАТЧІ (20 записів із балом >= 95, відсортованих за групою)
-----
ФЕП-31с  2  ids [192, 689]
ФЕП-32   6  ids [225, 262, 351, 353, 441, 812]
ФЕП-33   4  ids [149, 226, 719, 810]
ФЕП-34   6  ids [30, 218, 330, 520, 598, 765]
ФЕП-41   2  ids [220, 606]

accumulate / pairwise / takewhile / фільтр за групами
-----
оцінки перших шести:      [90.5, 45.8, 64.3, 80.1, 88.1, 60.0]
running_average:          [90.5, 68.15, 66.87, 70.18, 73.76, 71.47]
grade_changes:            [-44.7, 18.5, 15.8, 8.0, -28.1]
takewhile id <= 300, бал >= 90: 25 записів, ids [1, 27, 28, 30, 45, 51, 65, 77, 105, 116, 122, 142,
149, 166, 192, 218, 220, 225, 226, 246, 248, 251, 262, 265, 280]
in_groups(ФЕП-31с), бал >= 95, перші 3: [(192, 95.1), (689, 95.5), (1008, 96.3)]

chain: ОДИН ПОТІК ІЗ КІЛЬКОХ ФАЙЛІВ
-----
файли: ['students_10000.csv', 'students_100000.csv']
записів у ланцюжку з балом >= 85: 22 708

STREAMING EXPORT: студенти з балом >= 90 у новий файл
-----
data\honours.csv: 18 557 записів, 891 KB
перші 3 рядки: ['student_id,name,group,grade', '1,Кравець Максим,ФЕП-33,90.5', '27,Савчук Дарина,ФЕП-
41,94.6']

```

Потоковий експорт пише студентів із балом від 90 у новий CSV у міру надходження; міст `to_grade_records()` віддає той самий потік у формі `GradeRecord`, яку розуміє пакет `analytics` із ЛР2, тож функції попередньої роботи працюють над потоком без змін.

9. Експеримент: *eager* проти *lazy*

Обидві реалізації виконують ті самі стадії над тим самим файлом і дають той самий результат — різниця лише в стратегії обчислення. *Eager*-варіант повертає список на кожній стадії, тому в піку співіснують кілька копій даних; *lazy*-варіант — конвєсер із розділу 7, вичерпаний агрегатором. Час вимірюється без `tracemalloc`, пікова пам'ять — з ним (він сповільнює код у 2–3 рази, тому ці два виміри розділено); «1st» — час до першого результату.

```

def read_all_rows(path: Path) -> list[RawRow]:
    """Eager source: the whole file becomes a list of dicts before anything else happens."""
    with path.open("r", encoding="utf-8", newline="") as file:
        return list(csv.DictReader(file))

```

```

def eager_records(path: Path, minimum: float = MINIMUM) -> list[StudentRecord]:
    """Eager pipeline: every stage returns a full list, so several copies of the data coexist at the
    peak."""
    rows = read_all_rows(path)
    parsed = [_parse_or_none(row) for row in rows]
    valid = [record for record in parsed if record is not None]
    clean = [tidy(record) for record in valid]
    return [record for record in clean if record.grade >= minimum]

def eager_stats(path: Path, minimum: float = MINIMUM) -> Stats:
    return streaming_stats(eager_records(path, minimum))

# ----- lazy
def lazy_stats(path: Path, minimum: float = MINIMUM) -> Stats:
    """Lazy pipeline drained by the aggregation sink; one record is alive at a time."""
    return streaming_stats(build_pipeline(path, minimum))

def peak_memory(function: Callable[[], T]) -> tuple[T, int]:
    """(result, peak bytes allocated by Python objects). tracemalloc slows the code 2-3x, so elapsed
    time
    is always taken from a separate, untraced run."""
    tracemalloc.start()
    try:
        result = function()
        return result, tracemalloc.get_traced_memory()[1]
    finally:
        tracemalloc.stop()

@dataclass(frozen=True)

```

9.1. Час виконання

Записів	Eager, час	Lazy, час	Eager, до 1-го	Lazy, до 1-го
10 000	67.8 ms	71.3 ms	67.7 ms	449 µs
100 000	785.5 ms	764.5 ms	809.8 ms	799 µs
500 000	4.43 s	3.92 s	4.13 s	518 µs

9.2. Пікова пам'ять

Записів	Eager, peak	Lazy, peak	Відношення
10 000	7.94 MB	70 KB	116×

100 000	79.60 MB	70 KB	1 164×
500 000	398.61 MB	70 KB	5 831×

9.3. Раннє завершення на 1 000 000 записів

РАННЄ ЗАВЕРШЕННЯ НА 1 000 000 ЗАПИСАХ

```
lazy find_first: 63.5 ms, прочитано 6 988 рядків -> 6987 Кравець Марта ФЕП-33 99.5
eager список: 8.99 s, прочитано 1 000 001 рядків -> 6987 Кравець Марта ФЕП-33
99.5
```

9.4. Аналіз

Пам'ять. Eager росте лінійно — близько 0.8 KB на запис (список dict від DictReader плюс список записів), 400 MB на півмільйона рядків; lazy тримає сталі 70 KB незалежно від обсягу — це Stats, Counter за групами й кадри генераторів. Саме це і є «streaming»: файл може бути більшим за RAM.

Час. На 10 000 записів lazy на 5 % повільніший — кожен next() коштує перемикання кадру, а eager на малому обсязі виграє від щільних списків. Від 100 000 lazy швидший (на 500 000 — на 11.5 %), бо eager платить за виділення й копіювання великих списків, промахи кешу та роботу збирача сміття. Отже генератор не гарантує меншого загального часу; він гарантує сталу пам'ять.

Час до першого результату для eager дорівнює всій обробці (перший елемент доступний лише після повного списку), для lazy — під мілісекунду і не залежить від n. Це видно й на пошуку: find_first знайшов студента, прочитавши 6 988 рядків із мільйона, за 63.5 ms проти 8.99 s для eager-списку — у 140 разів швидше, і решту файлу ніхто не читав.

Ціна лінивості. Генератор одноразовий; помилки спливають під час споживання, а не при створенні (build_pipeline('missing.csv') створюється без винятку, FileNotFoundError летить на першому next()); len() та індексування недоступні; логування стадій не спрацьовує, якщо потік зупинили раніше. Демонстрація показує кожен з цих пасток окремо.

ТИПОВІ ПОМИЛКИ

```
вичерпаний генератор: list(g) -> [0, 1, 4, 9, 16] list(g) вдруге -> []
lazy exception: генератор створено без помилки; споживання -> ValueError: invalid literal for int()
with base 10: 'x'
build_pipeline('missing.csv') створено без помилки; next() -> FileNotFoundError лише під час
споживання
```

10. Тести та перевірка типів

Пакет покрито 23 тестами у tests/test_streaming.py (протокол iterator, one-shot, пізнє відкриття файлу, коди причин валідації, batching, рекурсивний flatten, збір streaming_stats зі стандартними функціями, раннє завершення пошуку, лінивість конвеєра до першого next(), збір

результатів eager і lazy, логування лише вичерпаних стадій). Разом із попередніми роботами — 44 тести; туру без зауважень.

```
..... [100%]
44 passed in 0.37s

Success: no issues found in 26 source files
```

11. Робота з Git

Роботу зафіксовано трьома комітами в гілці lab3 поверх main: стадії конвеєра; генератор даних, експеримент і демонстрація; тести, README та версія.

```
1082a54 Add streaming tests and README section, bump version to 0.3.0
1955d95 Add synthetic CSV generator, eager vs lazy experiment and streaming demo CLI
0485c47 Add streaming package: iterator protocol, generators and lazy pipeline stages
ec3e1ec Fix group name in README
9770176 Wire analytics demo into CLI, add tests, bump version to 0.2.0
94e46bd Add list vs dict search benchmark
3810ae2 Add analytics: averages, ranking, threshold closure, pipeline
28f41c3 Add collection processors: filtering, grouping, indexing, counting

On branch lab3
nothing to commit, working tree clean
```

12. Відповіді на контрольні запитання

1. Що таке iterable?

Об'єкт, який уміє видати iterator: має `__iter__()` (або `__getitem__` з цілими індексами) — list, str, файл, CsvDataset.

2. Що таке iterator?

Об'єкт із `__next__()`, який видає елементи по одному й кидає StopIteration, коли їх нема; його `__iter__()` повертає self.

3. Чим iterable відрізняється від iterator?

Iterable можна обходити багато разів (щоразу новий iterator); iterator — це сам стан обходу, одноразовий.

4. Для чого використовується iter()?

Отримати iterator з iterable — виклик `__iter__()`.

5. Для чого використовується next()?

Витягнути наступний елемент — виклик `__next__()`; другий аргумент повертається замість StopIteration.

6. Що робить __iter__()?

Повертає iterator для об'єкта; в самому iterator повертає self.

7. Що робить `__next__()`?

Повертає наступний елемент або кидає `StopIteration`.

8. Що таке `StopIteration`?

Виняток-сигнал «елементи закінчилися»; цикл `for` ловить його сам.

9. Як цикл `for` використовує `iterator protocol`?

Викликає `iter()` на об'єкті, потім `next()` у циклі, поки не отримає `StopIteration`.

10. Чому `iterator` часто називають `one-shot object`?

Він зберігає позицію і не вміє повертатися; після вичерпання порожній назавжди — `list(reader)` вдруге дає `[]`.

11. Чому `iter(iterator)` is `iterator`?

Бо `__iter__()` `iterator` повертає self, щоб `iterator` можна було передати всюди, де чекають `iterable`.

12. Чи можна повторно використати `exhausted generator`?

Ні, він у стані `GEN_CLOSED` назавжди; потрібен новий виклик `generator function`.

13. Що таке `generator function`?

Функція з `yield` у тілі; виклик не виконує тіло, а повертає `generator-об'єкт`.

14. Яку роль виконує `yield`?

Віддати значення споживачу й призупинити функцію, зберігши її стан.

15. Чим `yield` відрізняється від `return`?

`return` завершує функцію назавжди (генератор кидає `StopIteration`), `yield` лише призупиняє.

16. Коли фактично починає виконуватися `generator function`?

При першому `next()`, а не при виклику — `read_lines()` відкриває файл лише тоді.

17. Що зберігає `generator` між викликами `next()`?

Локальні змінні, позицію в коді, стан циклів і відкриті блоки `with`.

18. Що таке `generator expression`?

(вираз `for x in iterable`) — лінійний аналог `list comprehension` без побудови списку.

19. Чим `generator expression` відрізняється від `list comprehension`?

Comprehension будує весь список у пам'яті одразу; generator expression видає елементи по одному.

20. Що таке lazy evaluation?

Обчислення лише тоді, коли результат справді потрібен споживачу.

21. Що таке eager evaluation?

Обчислити все одразу й зберегти повністю — список на кожній стадії.

22. Які переваги lazy evaluation?

Стала пам'ять, час до першого результату в мікросекундах, раннє завершення, робота з нескінченними потоками.

23. Які недоліки lazy evaluation?

Одноразовість, відкладені помилки, немає len() та індексів, накладні витрати на кожен next(), складніше налагоджувати.

24. Що таке streaming?

Обробка даних порціями в міру надходження без завантаження всього набору.

25. Чому streaming важливий для великих files?

Файл може бути більшим за RAM; потік тримає лише поточний рядок — 70 KB проти 400 MB у експерименті.

26. Чому read() може бути проблемою для multi-GB file?

Читає весь файл в один рядок у пам'яті: MemoryError або swap.

27. Чому file object є iterable?

Має __iter__/_next__ і видає рядки по одному з буфера.

28. Для чого використовується yield from?

Делегувати генерування іншому iterable (parse_rows → DictReader) або собі рекурсивно (flatten).

29. Що таке itertools?

Модуль стандартної бібліотеки з ефективними лінівими будівельними блоками для iterator-ів.

30. Для чого itertools.count()?

Нескінченний лічильник; основа student_ids().

31. Чому count() потрібно обмежувати?

Інакше `list(count())` чи `for` без `break` ніколи не завершаться.

32. Для чого `islice()`?

Ліниво взяти зріз `iterator` (перші `n` або від `m` до `n`) без матеріалізації.

33. Для чого `chain()`?

Склеїти кілька `iterable` в один потік; `chain.from_iterable` — для потоку файлів.

34. Як працює `takewhile()`?

Віддає елементи, поки предикат істинний, і назавжди зупиняється на першому хибному (`head_by_id`).

35. Як працює `dropwhile()`?

Навпаки: пропускає елементи, поки предикат істинний, далі віддає все без перевірки.

36. Для чого `accumulate()`?

Накопичувальні суми або згортки — `running_average`.

37. Для чого `pairwise()`?

Сусідні пари (`a`, `b`), (`b`, `c`), ... — `grade_changes`.

38. Як працює `groupby()`?

Об'єднує сусідні елементи з однаковим ключем у групи.

39. Чому перед `groupby()` часто потрібне `sorting`?

Бо він групує лише сусідів; невідсортований потік дає багато дрібних груп з одним ключем.

40. Що робить `product()`?

Декартів добуток кількох `iterable` — усі комбінації по одному елементу з кожного.

41. Чим `combinations()` відрізняється від `permutations()`?

`combinations` не враховує порядок (пара {`a`, `b`} один раз), `permutations` — враховує.

42. Чому `combinatorial iterators` можуть бути небезпечними навіть при `lazy execution`?

Кількість елементів росте факторіально; лінивість не рятує, якщо споживач хоче їх усі.

43. Що таке `batching`?

Розбиття потоку на порції фіксованого або адаптивного розміру.

44. Які переваги `batching`?

Менше звернень до бази чи API, обмежена пам'ять на порцію, можливість паралельної обробки порцій.

45. Що таке lazy pipeline?

Ланцюжок генераторів, де кожен тягне з попереднього по одному елементу на вимогу споживача.

46. Які stages може містити pipeline?

Source, cleaning, parsing, validation, filtering, transformation, batching, aggregation/sink.

47. Що є source у streaming pipeline?

Звідки беруться елементи: файл, генератор, мережа, count().

48. Що є transformation?

Стадія, що змінює або відбирає елементи, нічого не зберігаючи — map/filter.

49. Що є sink?

Споживач, який вичерпує потік: sum, Counter, streaming_stats, export_csv.

50. Чи гарантує generator менший total execution time?

Ні: на 10 000 записів lazy був на 5 % повільніший; виграш з'явився на великих обсягах через відсутність копіювання списків.

51. Яка основна memory advantage generator?

Пам'ять не залежить від n — у польоті один елемент (70 KB для 500 000 рядків).

52. Що таке time to first result?

Час від запуску до першого елемента; для eager це вся обробка, для lazy — мікросекунди.

53. Чому lazy pipeline добре підтримує early termination?

Споживач просто перестає викликати next(); решту файлу ніхто не читає — 6 988 рядків із мільйона.

54. Чому exception у generator може виникати із затримкою?

Тіло виконується під час next(), а не при створенні; помилка спливає там, де споживають.

55. Чому list(generator) може бути проблемою?

Це повернення до eager: усе матеріалізується; для нескінченного потоку не завершиться.

56. Що трапиться при list(count())?

Нескінченний цикл із зростанням пам'яті до MemoryError.

57. Як generator може впливати на lifecycle file resource?

Файл відкритий, поки генератор призупинений; закриває його вичерпання або close(); забутий генератор тримає дескриптор до збирання сміття.

58. Як тестувати невеликий generator?

Порівняти list(gen) з очікуваним списком і перевірити поведінку next() після вичерпання.

59. Як тестувати infinite generator?

Брати islice(gen, n) або кілька next(); ніколи не list().

60. Коли краще використовувати list, а коли generator?

List — коли потрібні len, індекси, кілька проходів або обсяг малий; generator — для потоків, великих файлів і ранньої зупинки.

13. Висновок

Побудовано потоковий конвеєр обробки результатів студентів за варіантом 1: генератор даних на count + islice, власний iterator LineReader і контейнер CsvDataset, generator functions для читання, очищення, парсингу й валідації, yield from для делегування та рекурсії, generator expressions для трансформації, сім засобів itertools, batching двох видів, агрегація за один прохід і лінивий пошук із раннім завершенням. Експеримент на 10 000 – 500 000 записів показав головне: eager споживає пам'ять лінійно (до 399 MB), lazy — сталі 70 KB; час до першого результату падає з секунд до мікросекунд, пошук у мільйоні рядків прискорюється у 140 разів, а загальний час обробки при цьому не гірший, а на великих обсягах на 11 % кращий. Ціна — одноразовість генераторів і відкладені помилки, які треба знати й тестувати. Пакет покрито 23 тестами, проходить туру і зафіксований трьома комітами; наступна робота отримує від нього потоковий вхід для модуля імпорту.